

Module : Bases de Données

Informations générales sur le module :

Masse Horaire	140
Type	Théorique et Pratique (TD et TP)
Contenu	Partie1 : La méthode Merise - MCD : dictionnaire de données, entités, relations, cardinalités, GDF, les formes normales - MLD : règles de transformation - rétro-conception : MLD vers MCD Partie2 : Langage SQL/ Outils : SqlServer) - LDD - LMD - LCD Partie 3 : Utilisation du langage Transact-Sql (TSQL) pour compléter ou améliorer la base de données : - Les éléments de base du TSQL - Procédures Stockées, - Fonctions utilisateur - Déclencheurs - Curseurs - Sauvegarde et Restauration des bases de données

--- Partie 1 : La méthode MERISE en Bref -----

I- INTRODUCTION

Le système d'information (SI) est un ensemble organisé de ressources (matériels, humains) qui permet de collecter, stocker, traiter et distribuer de l'information au sein de l'organisation.

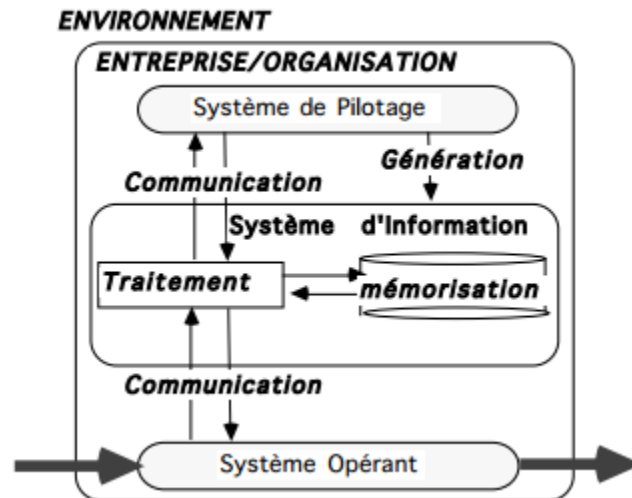


Figure 1 : Les fonctions du système d'information dans l'entreprise.

L'utilisation d'une méthode de conception permet de disposer d'un référentiel commun pour représenter les objets du système d'information étudié. Aussi, elle permet d'éviter les faiblesses du langage naturel (interprétation spécifique de chacun, ambiguïté des propos, etc).

Toute méthode informatique doit répondre à quatre objectifs principaux :

- définir ce que l'utilisateur final veut informatiser (quitte à lui faire comprendre ce qu'il veut), et sa faisabilité ;
- vérifier la cohérence de sa demande ;
- structurer les données à informatiser. Cela est primordial en informatique de gestion ;
- rester simple. La méthode doit rester un outil d'aide à la conception ou à la réalisation.

II- La Méthode MERISE

La méthode MERISE (**M**éthode d'**E**tude et de **R**éalisation Informatique pour les **S**ystèmes d'**E**ntreprises) est une démarche pour la modélisation de systèmes d'information. C'est une méthode franco-française, ses principaux auteurs sont : Arnold Rochfeld, Hubert Tardieu et José Morejon.

Comme toute méthode de conception de système d'information, MERISE adopte trois dimensions exprimant : la démarche ou cycle de vie ; le raisonnement ou cycle d'abstraction ; et la maîtrise ou cycle de décision. Pour plus de détails sur le cycle de vie et le cycle de décision de la méthode MERISE, je vous propose le lien intéressant suivant :

<https://merise.developpez.com/tutoriels/ingenierie-systemes-informations/?page=partie-1-principes-generaux-et-fondements-theoriques-de-la-methode-merise>

1. Le cycle d'abstraction de MERISE

Les étapes retenues dans Merise correspondent aux quatre niveaux d'abstraction : niveau conceptuel ; niveau organisationnel ; niveau logique ; et le niveau physique.

	Données	Traitements	
Niveau conceptuel	MCD <i>Modèle Conceptuel de Données</i>	MCT <i>Modèle Conceptuel de Traitements</i>	SIO <i>Système d'Information Organisationnel</i>
Niveau organisationnel	MOD <i>Modèle Organisationnel de Données</i>	MOT <i>Modèle Organisationnel de Traitements</i>	
Niveau logique	MLD <i>Modèle Logique de Données</i>	MLT <i>Modèle Logique de Traitements</i>	SII <i>Système d'Information Informatisé</i>
Niveau physique	MPD <i>Modèle Physique de Données</i>	MPT <i>Modèle Physique de Traitements</i>	

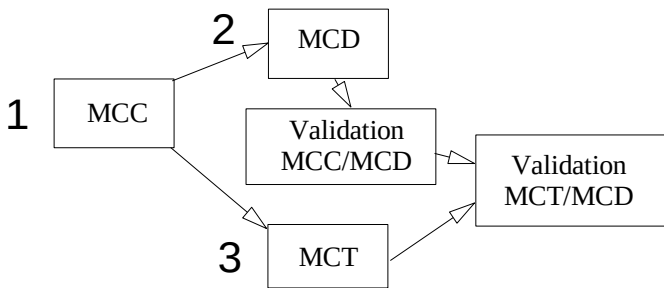
Figure 2: Les différents modèles du système d'information.

Les étapes principales d'une étude préalable sont, pour le niveau conceptuel :

- construction du modèle conceptuel de communication, domaines, partenaires et messages ;
- construction du modèle conceptuel de données ;
- validation des modèles de communication et de données ;
- construction du modèle conceptuel de traitement ;

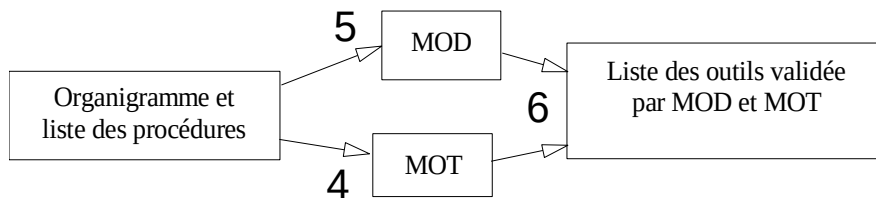
- validation des modèles conceptuels de données et de traitement.

Après le MCC, le MCD ou le MCT peuvent être construits.



Les étapes du niveau organisationnel et de définition des outils sont :

- construction de l'organigramme et de la liste des MOT ou procédures ;
- construction du ou des modèles organisationnels de données ;
- construction des modèles organisationnels de traitement ou procédures ;
- construction de la liste des outils validée par les modèles de données et de traitements.



Dans ce module, nous aborderons essentiellement la réalisation des modèles MCD et MLD.

2. Le MCD (Modèle Conceptuel de Données)

Le Modèle Conceptuel de Données ou MCD est le plus connu et le plus utilisé des éléments de la méthode Merise. **Il permet une représentation claire des données et des liens entre les données du domaine étudié.**

Un MCD a plusieurs buts :

- C'est un outil de **communication** : communication avec le donneur d'ordre, à l'intérieur d'une équipe de développement, avec les utilisateurs,
- C'est un outil de **normalisation** : passage de règles implicites et orales à des règles de gestion écrites, assimilation des systèmes parallèles, ...

La réalisation d'un MCD peut se dérouler en trois étapes :

Nous allons aborder ces étapes au travers d'une étude de cas simple. Le domaine étudié est un service Achat.

2.1 Constitution du dictionnaire des données

Cette étape consiste à référencer toutes les données du domaine. Pour notre service Achat, on peut recenser un certain nombre d'éléments :

- | | |
|----------------------------------|----------------------------|
| ▪ Nom du fournisseur | ▪ Condition de paiement |
| ▪ Date de la commande | ▪ Condition de livraison |
| ▪ Numéro de la commande | ▪ Téléphone du fournisseur |
| ▪ Quantité en stock de l'article | ▪ Adresse du fournisseur |
| ▪ Référence de l'article | ▪ Adresse de livraison |
| ▪ Prix unitaire de l'article | ▪ Unité de l'article |
| ▪ Désignation de l'article | ▪ Taux de TVA |
| ▪ Quantité commandée | |

Remarque :

Lors de l'établissement du dictionnaire, il peut s'y glisser des termes qui ne qualifient pas nécessairement des données. Il peut s'agir de traitements de données ou des flux d'information. Certaines expressions peuvent être polysémiques : elles recouvrent plusieurs sens, comme ici les conditions de paiement. D'autres expressions peuvent être à contrario synonymes. Ces problèmes peuvent être réglés dans les étapes suivantes de création du MCD. D'une manière générale, on essaiera de constituer un dictionnaire de données **exhaustif**.

Chaque terme du dictionnaire des données est appelé **Propriété**. L'étape suivant consiste à regrouper les **propriétés** dans des **entités**.

2.2 Entités

Dans notre cas, trois entités semblent se dégager :

- L'entité Fournisseur
- L'entité Article
- L'entité Commande

Une fois ces entités définies, on leur associe les propriétés du dictionnaire des données.

Un premier essai nous donne ceci :

Fournisseur	Article	Commande
Nom	Référence	Date
Adresse	Prix	Numéro
Téléphone	Désignation	
	Unité	

Une propriété ne peut être dans plusieurs entités. Ici, par exemple, le prix d'un article est une propriété de l'entité Article. Mais dans la mesure où ce prix est négocié en fonction de la commande, il sera nécessaire de disposer de deux propriétés prix : une propriété Prix Article dans l'entité Article et une propriété Prix Commande dans l'entité Commande.

Chaque fois qu'une propriété est associée à une entité on le signale dans le dictionnaire de données.

- F Nom du fournisseur
- C Date de la commande
- C Numéro de la commande
- Quantité en stock de l'article
- A Référence de l'article
- A Prix unitaire de l'article
- A Désignation de l'article
- Quantité commandée
- Condition de paiement
- Condition de livraison
- F Téléphone du fournisseur
- F Adresse du fournisseur
- Adresse de livraison
- A Unité de l'article
- Taux de TVA

On se rend compte que certains éléments du dictionnaire de données ne figurent pas dans les entités. Ceci peut être dû à plusieurs causes :

- Certains éléments ne sont pas des données. C'est peut être le cas des **conditions de paiement**. En effet, ces conditions peuvent être le résultat d'un calcul dépendant de la quantité commandée, du délai de livraison, ...
- Certains éléments ne sont pas dans le domaine étudié. C'est le cas de la **quantité en stock de l'article**. C'est bien une donnée mais cette donnée concerne la Gestion de stocks et non le service Achat.
- Certains éléments sont peut-être des propriétés d'entités que l'on n'a pas encore créées ou ils qualifient une **relation** entre entités. Par exemple la **quantité commandée** ne peut être une propriété de l'entité **Commande** que si une commande ne fait l'objet que d'un seul article, ce qui est un cas particulier. Généralement une commande

comporte plusieurs articles et il faudrait créer les propriétés **Quantité 1**, **Quantité 2**, **Quantité 3**, ... mais notre commande serait toujours limitée à un nombre N (ici 3) d'articles.

Commande	Commande
Date Numéro Quantité	Date Numéro Quantité 1 Quantité 2 Quantité 3

La prochaine phase est précisément la **création de relation entre les entités**.

Pour terminer la création des entités, il faut que chaque entité ait un identifiant. Un **identifiant** permet de **discriminer** chaque **occurrence** d'une entité. Tout identifiant aura donc une valeur unique. Par exemple, pour l'entité Article, l'identifiant est la Référence d'un article car deux articles différents ne peuvent avoir la même référence.

Référence	Prix	Désignation	Unité
01A12	12.45	Pince à vélo	2
01R11	105.00	Pneu	2

Un occurrence de l'entité Article est : 01A12,12.45,Pince à vélo,2.

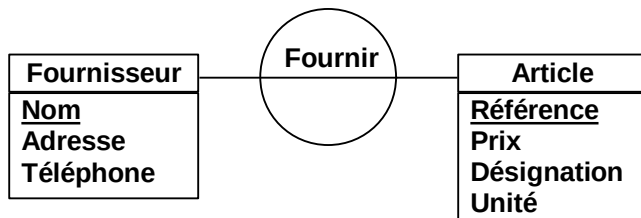
Pour distinguer les identifiants des autres propriétés, on a l'habitude de souligner la propriété constituant l'identifiant de l'entité.

Fournisseur	Article	Commande
<u>Nom</u> Adresse Téléphone	<u>Référence</u> Prix Désignation Unité	Date <u>Numéro</u>

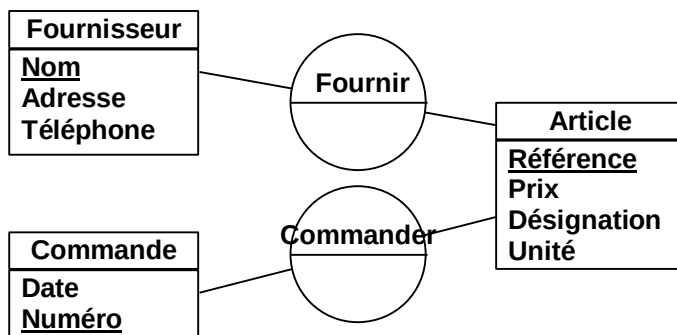
2.3 Relations

Une relation est un lien entre différentes entités. Ces liens se réalisent en se posant la question : 'quelle entité interagit sur cette ou ces entités ?'. Par exemple, un fournisseur fournit un article

et un article est fourni par un fournisseur. En effet il y a interaction entre les entités Fournisseur et Article que l'on traduit par l'action de Fournir.



De même, une commande comporte des articles et les articles appartiennent à une commande. La relation entre les entités Commande et Article est l'action de Commander.



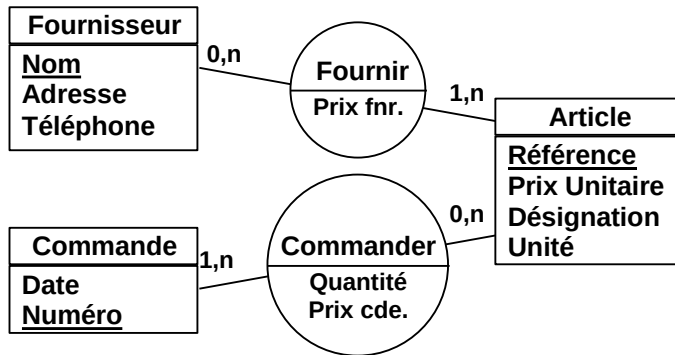
Une relation peut avoir des propriétés. C'est le cas ici de la propriété Quantité qui est une propriété de la relation Commander.

De même la possibilité pour les relations d'avoir des propriétés va nous permettre de créer de nouvelles données pour lever l'ambiguïté de la propriété Prix.

En effet la propriété Prix Unitaire est associée à l'entité Article, la propriété Prix commande est associée à la relation Commander et la propriété Prix fournisseur est associée à la relation Fournir.

2.4 Cardinalités

Notre MCD devient :



Les valeurs (0,n) ou (1,n) sont des cardinalités. Les cardinalités expriment le nombre minimum et maximum de fois qu'une entité est concernée par la relation.

Par exemple la cardinalité (0,n) de l'entité Fournisseur pour la relation Fournir signifie qu'un fournisseur peut fournir au moins zéro articles et au plus n articles. De même la cardinalité (1,n) de l'entité Article pour la relation Fournir signifie qu'un article est fourni par au moins un fournisseur et au plus N fournisseurs.

Plus intéressant est de constater que la relation Commander résout le problème évoqué lors de la création des entités et du rattachement de la donnée Quantité à l'entité Commande.

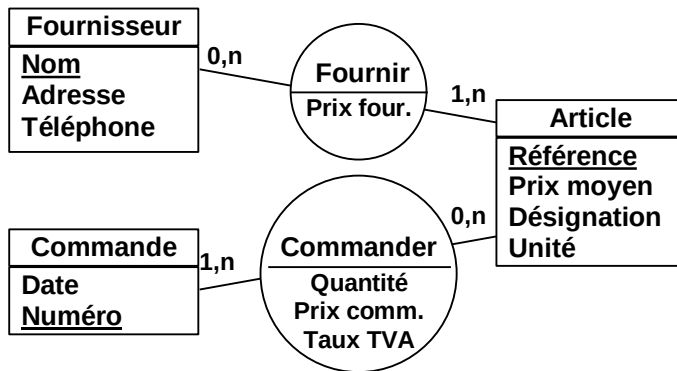
La cardinalité (1,n) de l'entité Commande pour la relation Commander signifie qu'une commande peut avoir au moins un article et au plus N (CQFD).

Si nous reprenons notre dictionnaire de données nous avons :

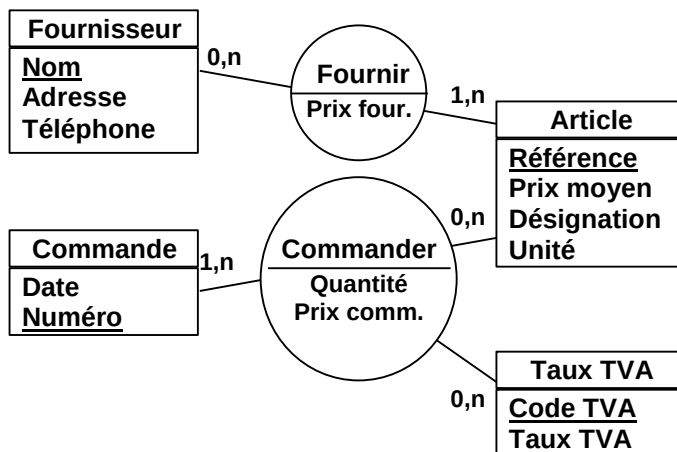
- F Nom du fournisseur
- C Date de la commande
- C Numéro de la commande
- (Quantité en stock de l'article, hors domaine)
- A Référence de l'article
- A Prix unitaire de l'article
- A Désignation de l'article
- CA Quantité commandée, propriété de la relation entre Commande et Article
- (Condition de paiement : traitement)
- (Condition de livraison : traitement)
- F Téléphone du fournisseur
- F Adresse du fournisseur
- Adresse de livraison
- A Unité de l'article
- Taux de TVA

Il nous reste l'Adresse de livraison et le Taux de TVA. L'Adresse de livraison est une propriété de l'entité Commande. Pour le taux de TVA, il y a deux façons de résoudre le problème :

- Soit rattacher le taux à la relation **Commander**



- Soit créer une entité Taux de TVA

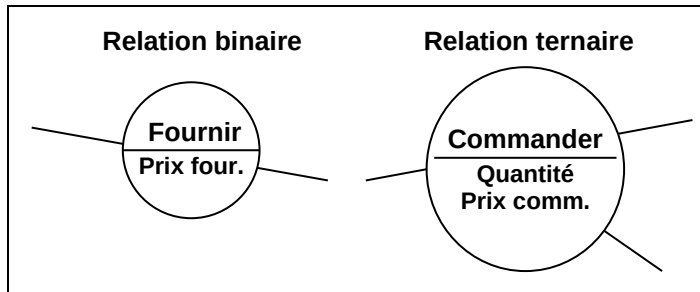


On ne peut, dans l'état actuel de nos connaissances, définir qu'elle est la présentation la mieux adaptée au problème. Il se peut de plus que la TVA soit fonction de la nature de l'article et/ou du lieu de livraison (Export, Régime Intra/Extra Communautaire).

Un MCD ne peut être unique et varie nécessairement en fonction de règles de gestion et de choix du concepteur.

Remarques :

- Une relation peut ne pas avoir de propriété
- Une relation peut être binaire, ternaire, ...



3. Normalisation :

Nous allons aborder dans ce document les principes de normalisation des données. Nous parlerons de modèles de données conformes à la 1^{ère}, 2^{ème} ou 3^{ème} forme normale.

Dans une [base de données relationnelle](#), une forme normale désigne un type de relation particulier entre les entités. Comme son nom l'indique, normale décrit la norme.

La normalisation des modèles de données permettra donc de s'assurer de la robustesse de la conception de la base pour améliorer la modélisation (et donc obtenir une meilleure représentation) et optimiser la mémorisation des données en évitant la [redondance](#) et les problèmes sous-jacents de mise à jour ou de cohérence.

La normalisation s'applique à toutes les [entités](#) et aux [relations](#) porteuses de propriétés.

On dira d'un modèle qu'il est en première, deuxième ou troisième forme normale. Il s'agit là des règles de normalisation les plus importantes, d'autres règles pouvant être appliquées mais de manière non systématique.

La forme normale vient après la simple validité d'un modèle relationnel, c'est-à-dire que les valeurs des différents attributs soient bien en dépendance fonctionnelle avec la clé primaire (complètement déterminés par la clé primaire).

En pratique, la première et la deuxième forme normale sont nécessaires pour avoir un modèle relationnel juste. Les formes normales supplémentaires ont leurs avantages et leurs inconvénients.

Les avantages sont :

- de limiter les redondances de données
- de limiter les incohérences de données qui pourraient les rendre inutilisables
- d'éviter les processus de mise à jour récurrents

Les inconvénients sont :

- des temps d'accès qui peuvent être plus longs si les requêtes sont trop complexes
- un manque de flexibilité au niveau de l'utilisation de l'espace de stockage

La troisième forme normale reste généralement une des meilleures solutions d'un point de vue architecture de base de données, mais pour des bases de données plus importantes, cela n'est pas toujours le cas. Il s'agit de choisir l'équilibre entre deux options :

- la génération dynamique des données via les jointures entre tables
- l'utilisation statiques de données correctement mises à jour

La normalisation des modèles de données a été popularisée principalement par la méthode Merise. La principale limite de la normalisation est que les données doivent se trouver dans une même base de données (dans un seul schéma).

Il convient d'être prudent lorsqu'on renonce à la forme normale. Il n'est pas garanti qu'une forme dénormalisée améliore les temps d'accès. En effet, la redondance peut entraîner une explosion des volumes de données qui peuvent écrouler les performances ou saturer les disques durs

3.1 La première forme normale (Notée 1FN) :

Respecte la première forme normale, la relation dont tous les attributs :

- contiennent une valeur atomique (les valeurs ne peuvent pas être divisées en plusieurs sous-valeurs dépendant également individuellement de la clé primaire). Ainsi, le nom et le prénom d'une personne doivent être dans des propriétés distinctes.
- contiennent des valeurs non répétitives (le cas contraire consiste à mettre une liste dans un seul attribut). On pourrait citer pour exemple une catégorie d'articles qui doit être définie dans une entité spécifique et non répétée dans l'entité article
- sont constants dans le temps (utiliser par exemple la date de naissance plutôt que l'âge).

Le non respect de deux premières conditions de la 1FN rend la recherche parmi les données plus lente parce qu'il faut analyser le contenu des champs. La troisième condition quant à elle évite qu'on doive régulièrement mettre à jour les données.

3.2 La deuxième forme normale (Notée 2FN) :

Respecte la seconde forme normale, la relation respectant la première forme normale et dont :

- tous les attributs non-clés ne dépendent pas d'une partie de la clé primaire mais bien de la totalité de la clé primaire.

Le non-respect de la 2FN entraîne une redondance des données qui encombrant alors inutilement la mémoire et l'espace disque.

3.3 La troisième forme normale (Notée 3FN) :

Respecte la troisième forme normale, la relation respectant la seconde forme normale et dont :

- tous les attributs n'appartenant pas à la clé ne dépendent pas d'un attribut non-clé. En d'autre terme, la dépendance fonctionnelle est directe.

4. Le MLD (Modèle Logique de Données)

Le MLD est une traduction du MCD .**Le MLD est l'expression du MCD dans le Système de Gestion des Bases de Données (SGBD) choisi.**

Remarque: Une base de données sur un domaine est un ensemble de données répondant aux trois critères :

- Exhaustivité : toutes les données nécessaires aux traitements sont stockées
- Structure : ces traitements peuvent accéder aux données dans des conditions acceptables
- Non redondance sémantique : les données ne sont pas répétées

Autrement dit, Le MLD est une étape intermédiaire pour passer du modèle E/A (Entité/Association), qui est un modèle sémantique, vers une représentation physique des données (SGBD relationnel).

Le MLD ne contient que des **tables** qui possèdent des attributs et une ou plusieurs clés. (clé primaire et étrangère). Toutes les tables ont **un nom unique**.

En général, il existe des règles de passage du MCD au MLD :

Règle 1 - Transformation d'un individu ou entité type :

Tout individu devient une **table**. Ses propriétés deviennent des attributs de la table (colonnes).

L'identifiant devient la clé primaire unique de la table.

Règle 2 – Association (*,1) - (*,n) :

La clé primaire de l'entité supérieure (côté plusieurs) devient attribut **clé étrangère** dans la table issue de l'entité inférieure (coté 1).

Remarque : Dans le cas d'une association « 1 à plusieurs » réflexive, ce nouvel attribut doit être renommé.

Règle 3 – Association (*,n) - (*,n) :

Devient une table. Les clés primaires des entités associées deviennent clés étrangères dans cette table. Les attributs de l'association deviennent attributs de la table. En général, la clé primaire de cette table est constituée de la concaténation des clés primaires des entités associées.

Remarque : il faut se demander si cette concaténation forme bien la clé primaire. Si ce n'est pas le cas, on peut essayer d'ajouter des attributs non-clés pour trouver la clé primaire. Ensuite, il faut se demander si on ne peut pas supprimer certains attributs clés étrangères pour réduire la clé primaire au minimum d'attributs.

Règle 4 – Association « 0.1 à plusieurs » : 2 possibilités :

- Si elles portent des attributs, on applique la règle 3 concernant les associations plusieurs à plusieurs. L'association donne une table.
- Si elles ne portent pas d'attributs, on applique la règle 2 concernant les associations 1 à plusieurs. Dans ce cas la clé étrangère produite n'est pas obligatoire puisque le minimum est à 0 (pas NOT NULL).

Règle 5 – L'héritage :

Dans le cas d'un héritage, chaque entité participante (espèce et genre) devient une table. La clé primaire de la table issue de l'entité genre devient clé étrangère dans les tables issues des entités espèces. Si une entité espèce n'a pas de clé primaire, la clé étrangère issue de l'entité genre devient la clé primaire de la table issue de l'entité espèce.

Règle 6 – Transformation d'une relation ternaire et supérieure

La relation donne lieu à la création d'une table qui aura comme clé primaire une clé composée des identifiants des individus sur lesquels porte la relation.

Remarque : Si plusieurs relations existent entre deux entités, on les traite séparément

Partie 2 : -- Création, Manipulation et Contrôle d'une Base de Données Relationnelle ---

L'objectif de cette partie est que le stagiaire soit capable d'alimenter une base de données relationnelle et d'en extraire les données avec le langage de requêtes SQL. Les requêtes doivent être écrites en langage SQL dans l'un des utilitaires du système de gestion de base de données en mode console. Pour les travaux pratiques, utiliser un SGBDR puissant tel que : Oracle ou SQL Server.

I- Modèle Relationnel

L'organisation des données au sein d'une BD a une importance essentielle pour faciliter l'accès et la mise à jour des données. Depuis leur apparition, les bases de données ont connu 4 évolutions majeures (Les bases de données hiérarchiques, Les bases de données réseau, Les bases de données relationnelles et Les bases de données objet). Aujourd'hui, les bases de données relationnelle (la 3ème évolution) est le type de base qui est le plus répandu.

Le modèle relationnel est basé sur l'**algèbre relationnel** et les travaux de E.F. Codd, il permet de modéliser facilement et sans grosse contraintes les systèmes du monde réel et de créer des bases de données simples à maintenir, à faire évoluer et indépendantes de leur support.

Remarque : Les avantages du modèle relationnel

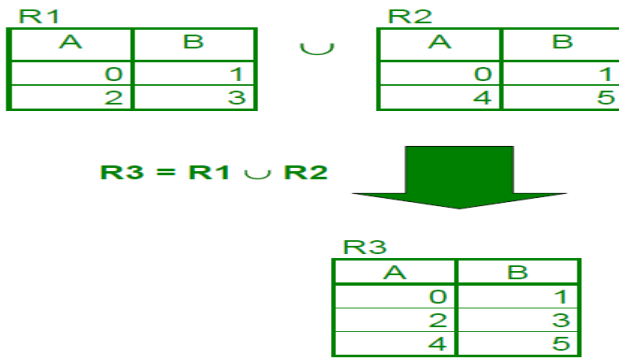
1. **Simplicité de présentation** : représentation sous forme de table
2. **Opérations relationnelles** : algèbre relationnelle, langage assertionnel (dans lequel une requête contient l'expression du résultat désiré)
3. **Indépendance physique** : optimisation des accès, stratégie d'accès déterminée par le système
4. **Indépendance logique** : concept de VUES
5. **Maintien de l'intégrité** : contraintes d'intégrité définies au niveau du schéma

II- L'algèbre relationnelle

L'algèbre relationnelle a été inventée par E. Codd comme une collection des opérations formelles qui agissent sur des relations et produisent des relations en résultats [Codd70]. Cette algèbre, qui constitue un ensemble d'opérations élémentaires associées au modèle relationnel, est sans doute une des forces essentielles du modèle. Les opérations de base peuvent être classées en deux types: les opérations ensemblistes traditionnelles et les opérations spécifiques.

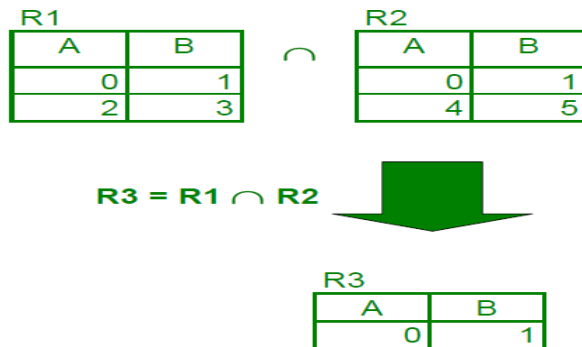
II.1 Union :

L'union de deux relations R1 et R2 de même schéma est une relation R3 de schéma identique qui a pour n-uplets les n-uplets de R1 et/ou R2



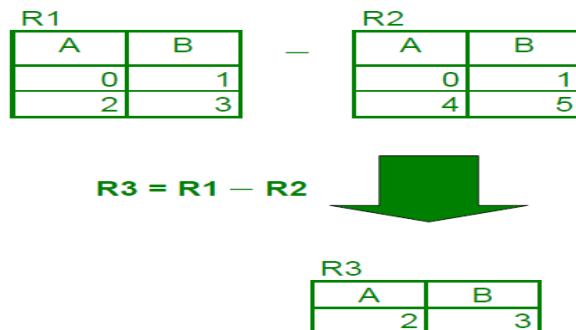
II.2 Intersection

L'intersection entre deux relations R1 et R2 de même schéma est une relation R3 de schéma identique ayant pour n-uplets les n-uplets communs à R1 et R2.



II.3- Différence

La différence entre deux relations R1 et R2 de même schéma est une relation R3 de schéma identique ayant pour n-uplets les n-uplets de R1 n'appartenant pas à R2.



II.4- Projection

La projection d'une relation R1 est la relation R2 obtenue en supprimant les attributs de R1 non mentionnés puis en éliminant éventuellement les n-uplets identiques. Elle correspond à un découpage vertical

Exemple : Requête: "Quels sont les références et les prix des produits ?"

PRODUIT (IdPro, Nom, Marque, Prix)



IdPro	Nom	Marque	Prix
P	PS1	IBM	1000
Q	Mac	Apple	2000
R	PS2	IBM	3000
S	Word	Microsoft	4000

 π PRODUIT (IdPro, Prix)


IdPro	Prix
P	1000
Q	2000
R	3000
S	4000

II.5 La restriction

La restriction d'une relation R1 est une relation R2 de même schéma n'ayant que les n-uplets de R1 répondant à la condition énoncée. Elle correspond à un découpage horizontal.

$$R2 = \sigma R1 (\text{condition})$$

La restriction d'une relation R1 suivant le critère "condition" où "condition" est une relation d'égalité ou d'inégalité entre 2 attributs ou entre un attribut et une valeur

Exemple : Requête: "Quels sont les produits de marque IBM?"

PRODUIT (IdPro, Nom, Marque, Prix)



IdPro	Nom	Marque	Prix
P	PS1	IBM	1000
Q	Mac	Apple	2000
R	PS2	IBM	3000
S	Word	Microsoft	4000

 σ PRODUIT (Marque = 'IBM')


IdPro	Nom	Marque	Prix
P	PS1	IBM	1000
R	PS2	IBM	3000

II.6 Produit Cartésien:

Formalisme : $R = \text{PRODUIT} (R1, R2)$

Cet opérateur porte sur deux relations. La relation résultat possède les attributs de chacune des relations d'origine et ses n-uplets sont formés par la concaténation de chaque n-uplet de la première relation avec l'ensemble des n-uplets de la deuxième.

Exemple :

Etudiants		Epreuves	
n°étudiant	nom	libellé épreuve	coefficient
101	DUPONT	Informatique	2
102	MARTIN	Mathématiques	3
		Gestion financière	5

Examen = PRODUIT (Etudiants, Epreuves)

n°étudiant	nom	libellé épreuve	coefficient
101	DUPONT	Informatique	2
101	DUPONT	Mathématiques	3
101	DUPONT	Gestion financière	5
102	MARTIN	Informatique	2
102	MARTIN	Mathématiques	3
102	MARTIN	Gestion financière	5

II.7 Jointure

La jointure de deux relations R1 et R2 est une relation R3 dont les n-uplets sont obtenus en concaténant les n-uplets de R1 avec ceux de R2 et en ne gardant que ceux qui vérifient la condition de liaison

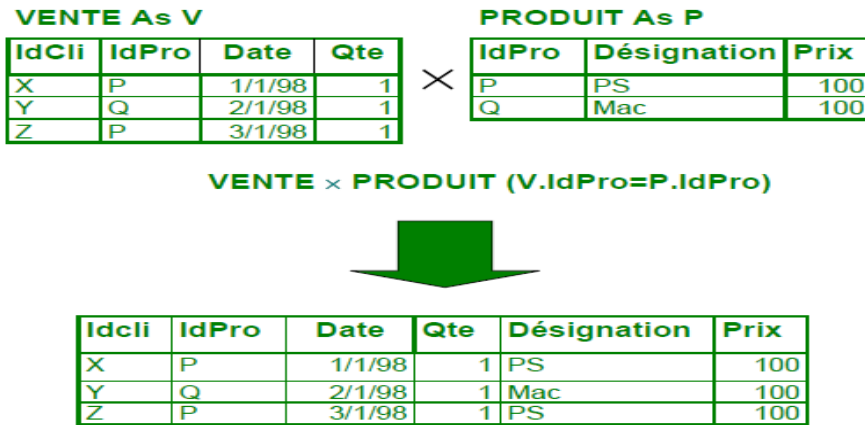
$R3 = R1 \times R2$ (condition)

Le schéma de la relation résultat de la jointure est la concaténation des schémas des opérandes (s'il y a des attributs de même nom, il faut les renommer). Les n-uplets de $R1 \times R2$ (condition) sont tous les couples (u1,u2) d'un n-uplet de R1 avec un n-uplet de R2 qui satisfont "condition".

La jointure de deux relations R1 et R2 est le **produit cartésien** des deux relations suivi d'une **restriction**.

Exemple

- Requête: *Donnez pour chaque vente la référence du produit, sa désignation, son prix, le numéro de client, la date et la quantité vendue.*



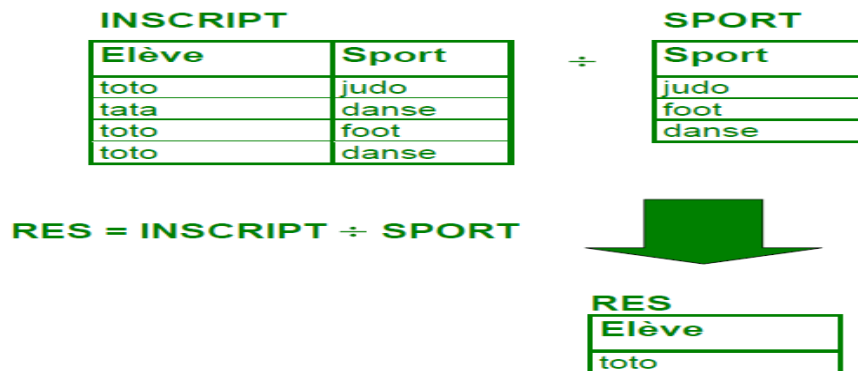
II.8 Division

La division permet de rechercher dans une relation les sous n-uplets qui sont complétés par tous ceux d'une autre relation

$$R3 = R1 \div R2$$

Elle permet de répondre à des questions qui sont formulées avec le quantificateur universel : "pour tout ..."

Exemple : Requête: *Quels sont les élèves qui sont inscrits à tous les sports?*



Le langage algébrique permet de formuler une question par une suite d'opérations de l'algèbre relationnelle

Exemple: Requêtes sur le schéma CLIENT, PRODUIT, VENTE

CLIENT (**IdCli**, **nom**, **ville**)

PRODUIT (**IdPro**, **désignation**, **marque**, **prix**)

VENTE (**IdCli**, **IdPro**, **date**, **qte**)

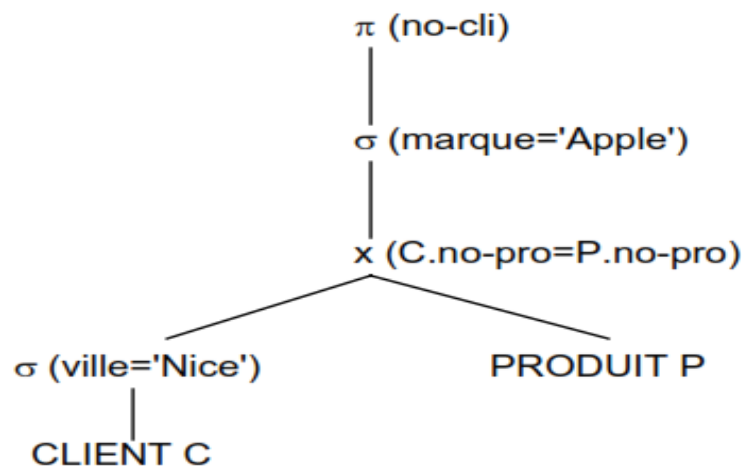
1- Donner les noms des clients qui ont acheté le produit 'p1'

- 2- Donner les numéros des clients ayant acheté un produit de marque Apple
- 3- Donner les numéros des produits de marque Apple et de prix < 5000 F
- 4- Donner les numéros des clients n'ayant acheté que des produits de marque Apple
- 5- Donner les numéros des clients ayant acheté tous les produits de marque Apple

Remarque : **Arbre algébrique**

Un arbre algébrique correspond à une décomposition de la requête en opérateurs élémentaires avec introduction de tables intermédiaires (: c'est donc l'équivalent d'une réécriture de la requête avec des vues.). Les feuilles de l'arbre représentent les tables de départ. La racine de l'arbre représente la table résultat. Tous les autres nœuds de l'arbre sont des **opérateurs** de l'algèbre relationnelle. Les arcs représentent les flux de données.

Exemple: Quels sont les clients de Nice ayant acheté un produit de marque 'Apple'



Plusieurs arbres équivalents peuvent être déduits d'un arbre donné à l'aide de règles de transformation simples, telles que permutation des jointures et restrictions, permutation des projections et des jointures, etc. Ces transformations sont à la base des techniques d'optimisation de requêtes

III- Langage des requêtes: SQL

S .Q L. signifie Structured Query Language. Il est issu de SEQUEL : Structured English Query Language, C'est le premier langage pour les S.G.B.D, Relationnels, Il a été développé par IBM en 1970 pour système R, son 1^{er} SGBDR. SQL a été reconnu par l'ANSI (Association de de Normalisation des Systèmes d'Information) puis s'est imposé comme norme.

S.Q.L. est un langage relationnel qui permet d'effectuer les tâches suivantes :

- ✓ Définition et modification de la structure de la base de données
- ✓ Interrogation et modification non procédurale de la base de données
- ✓ Contrôle de sécurité et d'intégrité de la base
- ✓ Sauvegarde et restauration des bases

SQL est un langage non procédural, il comporte plusieurs commandes qui se répartissent en trois familles fonctionnellement distinctes:

1. Langage de manipulation des données (**LMD**): sélectionner, insérer, modifier, ou supprimer des données dans une table.
2. Langage de définition des données(**LDD**): créer des tables dans une BDR, ainsi d'en modifier ou de supprimer leur structure.
3. Langage de contrôle de données(**LCD**): gérer la sécurité et les permissions au niveau des utilisateurs d'une BDR.

Remarque :

- Les ordres SQL peuvent être écrits indifféremment en majuscules et/ou minuscules.
- Les ordres SQL peuvent être écrits sur plusieurs lignes.
- Les mots clés ne doivent pas être abrégés ni scindés sur deux lignes différentes.
- Les clauses sont généralement placées sur des lignes distinctes.
- Les tabulations et indentations permettent une meilleure lisibilité.

1 :Langage de Manipulation des Données (LMD)

A.L'opération SELECT

L'opération select permet de rechercher des données à partir de plusieurs tables; le résultat est présenté sous forme d'une table réponse.

Syntaxe :

```
SELECT [distinct] * | <liste des champs >
FROM <Liste des tables>
[WHERE <critère de sélection >]
[GROUP BY <critères de regroupement>]
[HAVING <conditions de filtrage sur les groupes>]
[ORDER BY <liste des champs > ASC | DESC ] ;
```

Selon sa syntaxe, La requête SELECT la plus simple possible est la suivante :

```
SELECT * | <liste des champs >
FROM <Liste des tables>
```

Exemple : Soit la table : Produit(idp,libelle,marque,prix)

- 1- Sélection de Toutes les Colonnes : afficher tous les produits
select * from produit
- 2- Sélection d'une ou Plusieurs Colonnes Spécifiques (Projection) :

SELECT libelle, marque FROM PRODUIT

Remarque 1:

Il est possible de créer des expressions avec des données numériques et DATE au moyen d'opérateurs arithmétiques +, -, *, /

Exemple : select prix-200 from produit

Remarque 2 : La Valeur NULL :

NULL représente une valeur non disponible, non affectée, ou inconnue. NULL est différente du zéro ou de l'espace.

Remarque 3 : L'Alias de Colonne

Renomme un entête de colonne. il suit immédiatement le nom de la colonne ; le mot clé AS placé entre le nom et l'alias est optionnel. L'alias doit obligatoirement être inclus entre guillemets s'il contient des espaces, des caractères spéciaux ou si les majuscules/minuscules doivent être différenciées

1. Clause DINCTINCT

Contrairement à l'algèbre relationnelle, SQL n'élimine pas les doublons. Pour éliminer les doublons il faut spécifier **DINCTINCT**

Exemple: Donner les différentes marques de produit.

SELECT DISTINCT marque FROM PRODUIT

2. Clause WHERE (restriction)

Permet d'ajouter une condition à la requête de sélection. Avec la clause where, on peut utiliser :

- Opérateurs pour une condition simple:
- Opérateurs de comparaison: =, <>, <, >, <=, >=
- Prédicats : LIKE BETWEEN IN ALL EXISTS ANY IS NULL
- Opérateurs pour les conditions composées: AND, OR, NOT

L'opérateur BETWEEN

L'opérateur **BETWEEN**: teste si une valeur appartient à un intervalle.

Exemple: Lister les ventes dont la quantité est entre 4 et 12.

SELECT *
FROM VENTE
WHERE qte BETWEEN 4 AND 12

L'opérateur LIKE

L'opérateur **LIKE** : compare une chaîne avec un modèle

Caractères génériques

- % : Toute chaîne de 0 caractère ou plus
- _ : n'importe quel caractère
- [] : tout caractère de l'intervalle

- [^] : tout caractère en dehors de l'intervalle

Exemple: Donner la liste des clients dont le nom commence par a.

```
SELECT      *
FROM        CLIENT
WHERE nom LIKE 'a%'
```

L'opérateur IN

Teste l'appartenance à une liste de valeurs, il est utilisé avec les requêtes internes.

Exemple: Donner la liste des Clients des villes Rabat, Casa et Safi.

```
SELECT      *
FROM CLIENT
WHERE ville IN ('Rabat', 'Casa', 'Safi')
```

L'Opérateur IS NULL

Recherche de valeurs NULL avec l'opérateur IS NULL

Exemple : liste des produits de marque non affectées

```
SELECT *
FROM Produit
WHERE Marque IS NULL;
```

3. L'opération SELECT-jointure

Pour exprimer une jointure, on met la liste des tables à lier dans clause FROM. La condition de jointure sera affichée sur la clause WHERE.

Syntaxe:

```
SELECT liste des attributs
FROM liste des Tables (séparées par ,)
WHERE condition (y compris la condition de jointure)
```

Exemple: Afficher les noms des clients qui ont acheté les produits de libellé Clavier.

```
SELECT C.nom
FROM CLIENT C, PRODUIT P, VENTE V
WHERE C.numClt = V.numClt AND P.codeP = V.codeP AND P.libelle = 'Clavier'
```

TD1 : Voir énoncé

4. L'opération SELECT - les groupes

Clause GROUP BY

Une table peut être partitionnée selon un ou plusieurs critères. C'est le rôle de la clause **GROUP BY**. Elle permet de regrouper les enregistrements d'une requête suivant un ou plusieurs champs

Syntaxe:

```
SELECT liste des attributs
FROM Table
WHERE condition sur les lignes
GROUP BY critères de groupe
```

Exemple: Afficher le nombre de ventes par produit

```
SELECT codeP, COUNT (*)
```

```
FROM   VENTE
GROUP BY   codeP
```

Attention: Si on utilise GROUP BY, tous les attributs sélectionnés (de la clause SELECT) doivent apparaître sur la clause GROUP BY

❖ **Fonctions de calcul**

- ✓ MIN: la valeur minimale
- ✓ MAX: la valeur maximale
- ✓ COUNT: le nombre de valeurs
- ✓ AVG: la moyenne des valeurs
- ✓ SUM: la somme des valeurs

Exemple: Afficher la quantité totale vendue pour chaque produit

```
SELECT codeP, SUM(qte)
FROM   VENTE
GROUP BY   codeP
```

Clause Having

HAVING est une clause liée à la clause **GROUP BY**, qui permet d'éliminer les groupes qui ne vérifient pas la condition qu'elle définit.

Syntaxe:

```
SELECT  liste des attributs
FROM    Table
WHERE   condition sur les lignes
GROUP BY critères de groupe
HAVING  condition sur les groupes
```

Exemple: Afficher les codes des produits pour lesquels les quantités totales vendues sont supérieures à 100

```
SELECT      codeP, SUM(qte)
FROM VENTE
GROUP BY   codeP
HAVING      SUM(qte) > 100
```

Attention: les **conditions** sur les résultats des **fonctions de calcul** ne peuvent apparaître que sur la **clause HAVING**.

5. Clause ORDER BY

Pour trier les résultats d'une requête, on utilise la clause ORDER BY

Exemple : afficher les clients triés par ville

```
SELECT      *
FROM Client
ORDER BY   ville, adr ASC /DESC
```

6. L'opération SELECT-Sous requêtes

Exemple: Donner la liste des produits qui n'ont jamais été acheté.

Pour pouvoir répondre à cette question il faut tester que notre produit ne correspond à *aucune* ligne de la table vente. Ce qui n'est pas évident sans utiliser les sous requête.

Une sous-requête est une requête de sélection dont les résultats sont utilisés pour répondre à une autre requête.

4 Opérateurs principaux: IN, ALL, ANY, EXISTS

Réponse:

```
SELECT *
FROM PRODUIT
WHERE codeP NOT IN ( Select DISTINCT codeP From VENTE)
```

- **ALL** : vérifie la condition pour toutes les valeurs de la sous requête.
- **ANY** : vérifie si la condition est vraie pour au moins une valeur de la sous requête.
- **EXISTS**: vérifie si le résultat de la sous requête correspond à NULL ou pas.

Syntaxe:

```
SELECT      liste des attributs
FROM liste des tables
WHERE      Attr1 [>, <, >=, <=] ALL/ANY sous requête
```

TD2 : Voir énoncé

Résumé :

SELECT <i>liste_d'attributs</i>	←	{	DISTINCT, AS +, -, *, % AVG, MAX, MIN, SUM, COUNT
FROM <i>noms_de_tables</i>			
[WHERE <i>liste_de_critères</i>]	←	{	<, <=, >, >=, =, <> BETWEEN, IN, IS NULL LIKE, AND, OR, NOT
[GROUP BY <i>liste_d'attributs</i>]			
[HAVING <i>liste_de_critères</i>]			
[ORDER BY <i>liste_d'attributs</i>] ;			

B-Insertion

Pour ajouter un tuple (un enregistrement, une ligne) dans une table de la base de données, on procède comme suit :

Syntaxe :

```
INSERT INTO nom_table VALUES (valeur_attribut1, valeur_attribut2, ...);
Ou
INSERT INTO nom_table (attribut1, attribut2, ...) VALUES (valeur_attribut1,
valeur_attribut2, ..);
```

NB : NULL signifie l'absence de valeur

Exemple : Etant donné la table Etudiant (CNE, nom, prenom, ville). Si on souhaite insérer Les informations d'un nouvel étudiant disposant des informations suivantes (1234, Ben Salah, Salah, CASA), on écrit :

```
INSERT INTO ETUDIANT VALUES (1234, 'BEN SALAH', 'SALAH', 'CASA');
```

Définition d'une table par requête

Il est possible d'insérer dans une table des lignes provenant d'une autre table.

Syntaxe :

```
INSERT INTO nom_table (attribut1, attribut2,..) SELECT...
```

Le SELECT peut contenir n'importe quelle clause sauf un ORDER BY qui impliquerait un classement des lignes contraire à l'esprit du relationnel.

C-Modification

Pour modifier la valeur d'un attribut relatif à un ou plusieurs tuples d'une table, On procède comme suit :

```
UPDATE nom_table
SET attribut1 = valeur1, attribut2 = valeur2, ....
[WHERE condition];
```

Exemple : Etant donné la table Etudiant (CNE, nom, prenom, ville). Si jamais l'étudiant Salah Ben Salah habite maintenant à Tunis, on écrit dans ce cas :

```
UPDATE ETUDIANT
SET VILLE='TUNIS'
WHERE NCE=1234;
```

D-Suppression

Syntaxe : Il s'agit de supprimer un ou plusieurs tuples d'une table. Pour ce faire, on écrit :

```
DELETE FROM nom_table
[WHERE condition];
```

Exemple : Si on souhaite supprimer l'étudiant de NCE =1234 de la table Etudiant, on écrit :
DELETE FROM ETUDIANT
WHERE NCE=1234 ;

TD3 : voir énoncé

2. Langage de Définition des Données (LDD)

Le LDD est le sous langage SQL qui permet de définir les données dans un SGBDR.

A. Créer une base des données :

```
CREATE DataBase nom_BD ;
```

Exemple: CREATE DataBase GStagiaire;

Pour utiliser une base de données, on utilise la syntaxe : **Use nombd ;**

B. Création des tables :

1-type de données :

❖ Chaîne de caractères :

CHAR(N) : Déclare un champ chaîne de caractères de taille fixe (taille=n caractères)

VARCHAR(N) : Déclare un champ chaîne de caractères de taille variable (max=n caractères)

❖ ENTIER : SMALLINT INT LONG

❖ REEL : FLOAT

❖ NUMBER (N, M) : nombre de N chiffre, M chiffre après la virgule

❖ DATE : Date Time TimeStamp

Remarque : un champ auto incrémentale, on utilise la clause IDENTITY (grain, pas)

Grain : valeur de départ

Pas : valeur d'incrémententation

2-Créer une table :

Syntaxe : Pour créer une table, on fait recours à l'instruction suivante :

```
CREATE TABLE nom_de_la_table (
    nom_de_l'attribut type [ liste_de_contraintes_d'attribut ]
    nom_de_l'attribut type [ liste_de_contraintes_d'attribut ]
    ...
    liste_de_contraintes_de_table
);
```

où :

- $type \in \left\{ \begin{array}{l} \text{VARCHAR}(n) \text{ où } n \in \mathbb{N}, \text{ INT,} \\ \text{DATE, TIME} \end{array} \right\}$
- $contrainte_d'attribut \in \left\{ \begin{array}{l} \text{NULL, NOT NULL,} \\ \text{DEFAULT valeur,} \\ \text{CHECK(nom_de_l'attribut in domaine_de_définition)} \end{array} \right\}$
- $contrainte_de_table \in \left\{ \begin{array}{l} \text{PRIMARY KEY(liste_d'attributs),} \\ \text{FOREIGN KEY(nom_de_l'attribut) ...} \\ \text{... REFERENCES nom_de_la_table(nom_de_l'attribut)} \end{array} \right\}$

Contraintes d'intégrité :

- **Default** : Permet de spécifier une valeur par défaut pour un attribut de sorte qu'il ne sera jamais NULL
- **Not Null** : Cette contrainte oblige le SGBD de ne pas accepter NULL dans un attribut de la table
- **Unique** : Cette contrainte permet d'interdire la répétition des valeurs dans un attribut
- **Check** : si un attribut doit satisfaire une condition spécifique
- **Primary key** : crée une clé primaire pour la table.
- **Foreign key** : cette contrainte permet d'ajouter une clé étrangère à la table.

Remarque : On peut ajouter une contrainte au niveau d'attribut, comme on peut l'ajouter au niveau de la table

Syntaxe :

Contrainte d'attribut	Contrainte de table
Primary key	[CONSTRAINT nom_contrainte] PRIMARY KEY (attribut1, attribut2, ...)
-	CONSTRAINT nom_contrainte FOREIGN KEY (attribut_clé_étrangère) REFERENCES nom_table (attribut_référence)
Unique	CONSTRAINT nom_contrainte UNIQUE (colonne1, colonne2,...)
Check (condition)	CONSTRAINT nom_contrainte CHECK (condition)
Not null	

Exemple:

```
CREATE TABLE TPERSONNE
(PRS_ID INTEGER,
PRS_NOM VARCHAR(32),
PRS_PRENOM VARCHAR(32),
PRS_SEXE CHAR(1) DEFAULT 'M',
PRS_DATE_NAISSANCE DATETIME DEFAULT GETDATE(),
CONSTRAINT pk_personne PRIMARY KEY (PRS_ID)) ;
```

Exercice : créer les tables

Soit les tables : Module, Evaluation et Stagiaire
 Module (NumM, Intitule, Coef)
 Evaluation (#NumM, #CNE, DateN, Note)
 Stagiaire (CNE, Nom, Filiere)

TP1 : Voir énoncé

Remarque : Contraintes d'intégrité référentielle

SQL2 permet de spécifier les actions à entreprendre pour le maintien de l'intégrité référentielle, lors d'une suppression ou d'une modification d'un tuple référencé.

CASCADE : cascader les suppressions ou modifications

SET NULL : rendre nul les attributs référençant

RESTRICT : rejet de la mise à jour. c'est l'option par défaut

Au cours de déclaration des clés étrangères, On ajoute : on delete cascade /set null

Et

on update cascade/set null

Remarque :

ON DELETE CASCADE : indique que si une ligne est supprimée de la table maître, les lignes dépendantes de la table détail seront également supprimées.

Résumé

```
CREATE TABLE table
( définition
des colonnes
colonne type [ NOT NULL [UNIQUE] ]
[ DEFAULT valeur ]
[ PRIMARY KEY ]
[ REFERENCES table ]
[ CHECK condition ]
...
contraintes
de table
[ PRIMARY KEY (liste de colonnes) ],
[ UNIQUE (liste de colonnes) ]
...
[ FOREIGN KEY (liste de colonnes) REFERENCES table
[ ON DELETE {RESTRICT | CASCADE | SET NULL} ]
[ ON UPDATE {RESTRICT | CASCADE | SET NULL} ] ,
...
[ CHECK condition ]
... )
```

C. Modification du schéma d'une table :

1. Ajouter une colonne à une table :

Pour ajouter la colonne à une table :

Alter Table nom_table ADD nom_colonne type contrainte_d'attribut ;

2. Supprimer une colonne d'une table :

Pour supprimer la colonne d'une table

Alter Table nom_table DROP column nom_colonne ;

3. Changer le type des données d'une colonne :

Pour changer le type de colonne :

Alter Table nom_table Alter column nom_colonne type

4. Ajouter une contrainte à une table :

Pour ajouter une contrainte à une table :

Alter Table nom_table ADD Constraint nom_contrainte check(condition) ;

Alter Table nom_table ADD Constraint nom_contrainte unique(colonne1,colonne2) ;

5. Supprimer une contrainte d'une table :

Pour supprimer une contrainte d'une table :

Alter Table nom_table DROP Constraint nom_contrainte ;

6. Renommer le nom d'une colonne :

Pour changer le nom de la colonne :

Exec sp_rename 'nom_table.ancien_colonne','nouveau_nom','Column'

D .Supprimer des tables et des bases de données :

Pour supprimer les tables et les bases de données, on utilise la clause DROP

DROP TABLE nom_table

DROP DataBase nom_base_de_données

Exemple:

Soit le Schéma relationnel GestAud suivant:

Auditeur (#IdAud, Nom)

Examen (#IdEx, Salle, DateEx)

Passe_Ex (IdAud, IdEx,Note)

1. Créer la base de données GestAud

2. Créer les tables Auditeur, Examen, Passe_Ex (la valeur par défaut pour la note est la valeur 0)

3- Ajouter le champ telAud avec le type char(10)

- Modifier le type de champ telAud en varchar

-Renommer le champ telAud en téléphone

-Supprimer le champ téléphone de la table Auditeur

-Supprimer la contrainte par défaut 0 pour le champ note

-Modifier le champ note de telles sortes qu'il ne sera jamais Null

-Ajouter la contrainte pour que la valeur de la note soit entre 0 et 20

-Modifier le champ IdEx de la table Examen de sorte qu'il sera Null

-Ajouter la contrainte (clé primaire) pour les différentes tables

-Supprimer la contrainte de clé primaire pour la table Examen

-Ajouter la contrainte (clé étrangère) pour les différentes tables (ajouter la contrainte de clé primaire pour la table examen)

4-insérer des enregistrements pour les différentes tables de la base de données GestAud

E -les vues et les index

1 -vue

1.1-définition

Une vue est une table virtuelle dont la définition s'appuie sur des tables ou d'autres vues. Elle affiche des données qui ne lui sont pas propres, mais c'est comme une fenêtre par laquelle il est possible de visualiser ou de modifier des données venant des tables. Une vue est stockée en mémoire

1.2-Syntaxe

❖ Création d'une Vue

```
CREATE VIEW [ schema_name. ] view_name
AS select_statement
[ WITH CHECK OPTION ] [ ; ]
```

▪ WITH CHECK OPTION.

Cette option impose que toutes les instructions de modification de données exécutées sur la vue respectent le critère défini dans select

❖ Modifier une vue : Alter

```
ALTER VIEW [ schema_name . ] view_name
AS select_statement
[ WITH CHECK OPTION ] [ ; ]
```

❖ Suppression d'une Vue

L'ordre DROP VIEW view_name : permet de supprimer une vue.

La suppression d'une vue n'a aucun effet sur les tables sur lesquelles la vue est basée.

Exemple :

```
CREATE VIEW EMP10 AS
SELECT * FROM EMP
WHERE DEPT = 10
Select * from EMP10
```

2- Index

Un index est formé de clés auxquelles SQL peut accéder très rapidement. Comme pour l'index d'un livre, ces clés permettent de lire ensuite directement les données repérées par les clés. Un index permet de rendre la recherche sur une colonne ou plusieurs plus rapide.

2.1-CREATE Index

Un index se crée par la commande CREATE INDEX :

```
CREATE [UNIQUE] INDEX nom-index ON table (col1, col2,...)
```

On peut spécifier par l'option _UNIQUE_ que chaque valeur d'index doit être unique dans la table.

2.2-DROP INDEX

Un index peut être supprimé par la commande DROP INDEX :

```
DROP INDEX nom_index [ON table]
```

F- Notion de Transaction

1- Définition

Une transaction est une unité logique de traitement qui est : soit complètement exécutée, soit complètement abandonnée

Exemple : on suppose qu'on veut transférer une somme S d'un compte C1 vers un compte C2. Donc , on doit ajouter au compte C2 la somme S et en même temps on doit enlever du compte C1 la somme S.

Début_transaction

C1 := C1 - S

C2 := C2 + S

Fin_transaction

Cette transaction est constituée de deux 'actions élémentaires, mais elle doit être traitée comme une seule opération. Autrement dit le gestionnaire des transactions doit assurer que toutes les actions de la transaction sont exécutées, ou bien qu'aucune ne l'est.

2- La vie d'une Transaction

- **Vie normale** : La transaction s'exécute normalement jusqu'à la fin. Elle se termine par une instruction de validation **COMMIT** en SQL. Cette transaction est donc validée. Toutes les modifications faites sur la base de données par cette transaction sont considérées comme définitives.
- **Un assassinat** : Un événement extérieur vient interrompre l'exécution de la transaction. Cet arrêt peut provenir, soit d'une panne, soit d'une action délibérée de la part du SGBD qui décide de supprimer telle ou telle transaction (c'est le cas lorsqu'il détecte un interblocage).
- **Un suicide** : Au cours de son exécution, la transaction détecte certaines conditions qui font que la poursuite de son exécution s'avère impossible, elle peut se supprimer en exécutant une instruction d'annulation **ROLLBACK** en SQL

Dans ces deux derniers cas, tout doit se passer comme si la transaction n'avait jamais existée (toute trace d'exécution de la transaction est supprimée) : la transaction a été annulée.

3- Les propriétés des transactions

Un système de gestion transactionnel doit garantir les propriétés suivantes (résumées par le vocable ACID) :

- **Atomicité** (indivisible ou indécomposable) : Une transaction doit effectuer toutes ses mises à jour ou rien faire du tout
- **Cohérence** : La transaction doit faire passer la base de données d'un état cohérent à un autre.
- **Isolation** : Les résultats d'une transaction ne doivent être visibles aux autres transactions qu'une fois la transaction validée
- **Durabilité** : Dès qu'une transaction valide ses modifications, le système doit garantir que ces modifications seront conservées en cas de panne

4- Créer une Transaction

- Début d'une transaction : **BEGIN TRANSACTION (ou BEGIN)** ;

Remarque : Cette instruction est optionnelle (voire inconnue de certains SGBD), une transaction étant débutée de façon implicite dès qu'une instruction est initiée sur la BD

- Fin correcte d'une transaction : **COMMIT TRANSACTION (ou COMMIT)** ;

Cette instruction SQL signale la fin d'une transaction couronnée de succès. Elle indique donc au gestionnaire de transaction que l'unité logique de travail s'est terminée dans un état cohérent est que les données peuvent effectivement être modifiées de façon durable.

- Fin incorrecte d'une transaction : **ROLLBACK TRANSACTION (ou ROLLBACK)** ;

Cette instruction SQL signale la fin d'une transaction pour laquelle quelque chose s'est mal passé. Elle indique donc au gestionnaire de transaction que l'unité logique de travail s'est terminée dans un état potentiellement incohérent et donc que les données ne doivent pas être modifiées en annulant les modifications réalisées au cours de la transaction.

Remarque : Combien de tables sont créées par cette série d'instruction ?

```
SET AUTOCOMMIT OFF
CREATE TABLE T1 (A INTEGER);
CREATE TABLE T2 (A INTEGER);
CREATE TABLE T3 (A INTEGER);
INSERT INTO T1 VALUES (1);
INSERT INTO T2 VALUES (1);
INSERT INTO T3 VALUES (1);
ROLLBACK;
```

Réponse : 3

Les transactions n'affectent normalement que les instructions LMD et non les instructions LDD. Notons néanmoins que certains SGBD peuvent être paramétrés pour autoriser certaines instructions LDD à être affectées par les transactions (ce qui est un détournement de l'usage théorique des transactions).

5- Mécanisme interne aux transactions

Tant qu'on ne fait pas de COMMIT/ROLLBACK, toutes les instructions internes à la transaction sont stockées en mémoire (SQL server trace celles-ci dans son fichier journal .ldf).

Une fois toutes les instructions de la transaction terminées, et qu'aucun COMMIT/ROLLBACK n'a été effectué => il n'y a toujours pas d'écriture des données dans la base à proprement parler (dans le fichier .mdf).

A fréquence régulière, SQL Server via un algorithme interne vérifie le journal de transaction de façon répétitive à des moments opportuns (en arrière plan), depuis le dernier point de lecture (son dernier CheckPoint)

=> à ce moment précis :

- S'il constate la présence de "BEGIN TRANSACTION" non commité, il va actionner le COMMIT et donc l'écriture sur le disque (dans le .mdf).
- Si, par contre, il constate qu'il n'y a pas de COMMIT en mémoire (pas d'instructions COMMIT tracées dans le fichier.ldf), il supprimera la transaction correspondante en mémoire. Ce qui correspond en effet au respect de sa caractéristique Atomic.

C'est grâce à ce mécanisme que SQL Server, après un crache système, sait reprendre et terminer (ou non) des transactions.

TP : voir énoncé

3. Langage de Contrôle des Données (LCD)

Un SGBD est équipé des mécanismes qui assurent la sécurité des données. Ainsi, on peut permettre ou interdire l'accès à des données. Des mécanismes du SGBD empêchent des personnes non autorisées de lire ou de modifier des données non autorisées.

Pour créer des utilisateurs sur une base de données, il faut créer des connexions (autant de connexions que l'utilisateur)

1-Créer une connexion

Create login nom_connexion with password= '1j@llp.23' DEFAULT_DATABASE=nom_BD

2-Créer des utilisateurs

Create user nom_utilisateurs for login nom_connexion

3-Attribuer des autorisations à un utilisateur

Grant select, insert, update, delete ON nom_table To nom_utilisateurs

4- Interdire des autorisations à un utilisateur

Deny insert,select,update,delete ON nom_table to nom_utilisateurs

5-Retirer l'autorisation

Revoke insert,select,update,delete ON nom_table to nom_utilisateurs

6-Creation des roles (groupements de droits):

Create Role nom_Role AUTHORIZATION nom_utilisateurs

Exec sp_addrolemember 'nom_Role' , 'nom_utilisateurs'

Remarques : rôles prédéfinis

- Principaux rôles fixes de serveur :

<i>Rôle</i>	<i>Autorisation</i>
sysadmin	Exécution de toutes les activités
dbcreator	Création et modification des bases de données
processadmin	Gestion des processus SQL Server
serveradmin	Configuration des paramètres de serveur
setupadmin	Gestion de la réplication
securityadmin	Gestion des connexions et autorisations

- Principaux rôles fixes de BD :

<i>Rôle</i>	<i>Autorisation</i>
public	Conservation de toutes les autorisations par défaut
db_owner	Exécution de toutes les activités sur une base
db_accessadmin	Ajout ou suppression d'utilisateurs, groupes et rôles
db_ddladmin	Ajout, modif ou supp. d'objets de base de données
db_security admin	Affectation d'autorisations d'instruction et sur les objets
db_backupoperator	Sauvegarde et restauration de la base de données
db_datareader	Lecture des données dans toutes les tables
db_datawriter	Ajout, modification ou suppression des données
db_denydatareader	Impossible de lire les données
db_denydatawriter	Impossible de modifier les données

Exemple:

- 1- Créer trois utilisateurs a1(123), a2(321), a3 (abc)
- 2 -Créer trois rôles Surveillant, Formateur, Directeur
- 3- Donner le droit de modification (insert, update, delete)
 - sur Etudiant par Surveillant
 - sur Matière par Formateur
 - sur Evaluer par Directeur
- 4- Ajouter des utilisateurs
 - a1 dans Surveillant
 - a2 dans Formateur
 - a3 dans Directeur

Annexe :

Auto—jointure

L'autojointure est la jointure entre une table et elle-même, pour sélectionner des enregistrements correspondant à d'autres de la même table. Il est nécessaire de recourir alors à des alias pour définir la table déjà utilisée.

Exemple : Liste des avions de même capacité

```
SELECT AVION.AV#, AVION.MARQUE, AVION.TYPE,AVION.CAPACITE,
AV2.AV#, AV2.MARQUE, AV2.TYPEAVION, AVION.CAPACITE
FROM AVION, AVION AS "AV2"
ON AVION.CAPACITE = AV2.CAPACITE
WHERE AVION.AV# > AV2.AV#
```

A noter : L'expression de la clause WHERE pour éliminer les paires de lignes figurant plusieurs fois.

Exemple : Donner les noms des clients de la même ville que John

```
SELECT C2.nom FROM client C1 , client C2
WHERE C1.ville = C2.ville AND C1.nom = 'John' AND C2.nom <> 'John'
```

Cet exemple utilise, pour le couplage des villes, la jointure de la table Client avec elle-même (autojointure)

Pour pouvoir distinguer les références ville dans les 2 copies, il faut introduire 2 alias différents C1 et C2 de la table client

Jointures externes

Dans le cas d'une jointure interne, lorsqu'une ligne d'une table ne satisfait pas la condition de jointure, cette ligne n'apparaît pas dans le résultat final.

Il peut, cependant, être souhaitable de conserver les lignes d'une table (left ou right) qui ne répondent pas à la condition de jointure. On parle alors de jointure externe (outer join).

Syntaxe de la jointure externe :

```
SELECT ...
FROM <table gauche>
    LEFT | RIGHT | FULL OUTER JOIN <table droite>
    ON condition de jointure
```

- syntaxe jointure interne :

```
SELECT ...
FROM <table gauche>
    [INNER]JOIN <table droite>
    ON <condition de jointure>
```

Opérateurs ensemblistes

Union

L'opérateur UNION permet de fusionner deux sélections de tables pour obtenir un ensemble de lignes égal à la réunion des lignes des deux sélections. Les lignes communes n'apparaîtront qu'une fois.

Exemple : Liste des ingénieurs des deux filiales

```
SELECT attribut1, attribut2, ... FROM table1
UNION
SELECT attribut1, attribut2, ... FROM table2 ;
```

Différence

L'opérateur EXCEPT permet d'ôter d'une sélection les lignes obtenues dans une deuxième sélection.

Exemple : Liste des départements qui ont des employés dans la première filiale mais pas dans la deuxième

```
SELECT attribut1, attribut2, ... FROM table1
```

EXCEPT

SELECT attribut1, attribut2, ... **FROM** table2 ;

ou

SELECT attribut1, attribut2, ... **FROM** table1 **WHERE** attribut1 **NOT IN** (**SELECT** attribut1 **FROM** table2) ;

Intersection

L'opérateur INTERSECT permet d'obtenir l'ensemble des lignes communes à deux interrogations.

Exemple : Liste des départements qui ont des employés dans les deux filiales

SELECT attribut1, attribut2, ... **FROM** table1

INTERSECT

SELECT attribut1, attribut2, ... **FROM** table2 ;

ou

SELECT attribut1, attribut2, ... **FROM** table1

WHERE attribut1 **IN** (**SELECT** attribut1 **FROM** table2) ;

Partie 3 : -- Transact-SQL : Procédures stockées, Fonctions Utilisateur, Déclencheurs, Curseurs—

Définition : Transact-SQL est une extension de SQL vers un langage de programmation, il est utilisé par Microsoft SQL Server

I- Les éléments de base

1. Les variables :

1.1 Déclaration :

Pour déclarer des variables, on peut utiliser l'une des méthodes suivantes :

- Déclaration d'une variable par ligne : **Declare @nom_variable type**
- Déclaration multiple : **Declare @var1 type1, @var2 type2**

Exemple: Declare @maVariable int
 Declare @x int, @y int, @z char
 // NB : Declare @x ,@y int est incorrecte

1.2 Affectation de valeur :

Pour donner une valeur à une variable, on peut utiliser select (aussi pour une affectation multiple) ou set (pour une seule variable).

Syntaxe générale : Select @variable=Expression où set @variable=Expression

Exemple 1: affectation d'une variable

```
select @i=3 ;                set @j=4 ;                select @str='TSDI'
```

Exemple 2 : Affectation multiple

```
select @i=3,@j=4,@str='TSDI' ;
```

NB : set @i=3, @j=4, @str='TSDI' est une affectation incorrecte.

1.3 Affichage des valeurs :

Pour afficher le contenu d'une variable on utilise la même instruction **select**.

Exemples : Select @i ;

```
Select @i,@j,@str // Affichage multiple
```

Remarque 1 : On peut utiliser select pour affecter une valeur ou bien pour afficher une autre, mais pas pour faire les deux, donc l'instruction select @i=20, @str est incorrecte.

Remarque 2 : Affichage avec print

On peut utiliser la commande print pour afficher un résultat sous format message. Syntaxe : Print 'Chaine de caractère'

Dans l'analyseur de requête SQL on a deux sortie d'affichage Messages pour Print et Table pour Select

Exemple de variable de type table :

```
Declare @stg table (numInsc int primary key, nom varchar(20), moyenne float)
/*la particularité des variables de type table, est qu'on peut utiliser des commandes insert, select,
update, delete */
insert into @stg values(103,'Jalimi',12.2),(107,'ibrahimi', 15.06)
insert into @stg values(200,'SOQRAT',10.89)
Select * from @stg
Select avg(moyenne) from @stg
```

2. Les Variables Globales :

Les variables globales sont affectées directement par le serveur, elle retournent une seule valeur, elles sont utilisées pour communiquer une information au client, elle sont notées **@@nom_variable**

Exemples :

Variable globale	Rôle
@@error	indique le type d'erreur survenu lors de la dernière instruction.
@@rowcount	indique le nombre de lignes affectées par la dernière instruction.
@@identity	indique la valeur affecté à un attribut avec la propriété identity

3. Structure Alternative et répétitive :

3.1 Structure Alternative :

La structure alternative est une structure de contrôle qui permet d'exécuter un de deux actions suivant une condition.

Syntaxe :

<pre>If(condition) -instruction ou bloc d'instruction else -instruction ou bloc d'instruction</pre>

Avec :

- Chaque Bloc d'instructions commence par begin et se termine par end .
- La partie « else » est optionnelle. Il est possible d'imbriquer des if.

Exemple :

```
Declare @stg table(numInsc int primary key, nom varchar(20))
insert into @stg values(103,'LAAROUSSI','SALAH',14)
If not exists(select * from @stg ) Print 'la table est vide'
Else Print 'la table n'est pas vide'
```

3.2 Structure Itérative :

La structure itérative est une structure qui permet d'exécuter un même traitement plusieurs fois.
Syntaxe générale :

```
While(condition)
-instruction ou bloc d'instructions
```

Exemple : calcule de la factorielle d'un nombre

```
Declare @i int, @f int,@n int
select @n=6, @f=1, @i=1
while (@i<=@n)
begin
set @f=@f*@i
set @i=@i+1
end
select @f as "le factoriel"
```

4. Structure de choix (CASE) :

L'instruction CASE, permet d'attribuer des valeurs en fonction d'une condition

Remarque : La fonction CASE est une expression Transact-SQL spéciale qui permet l'affichage d'une valeur de remplacement en fonction de la valeur d'une colonne. Ce changement est temporaire. Par conséquent, aucune modification permanente n'est apportée aux données.

Exemple : afficher le nom complet de la ville :

```
SELECT nom, CASE ville
                WHEN 'CA' THEN 'Casablanca'
                WHEN 'Kn' THEN 'Kenitra'
                WHEN 'RB' THEN 'Rabat'
                END
FROM client ORDER BY nom ;
```

Exercice : soit la table stagiaire (numi, noms, prenom,note)

Afficher le nom et le prenom de chaque stagiaire ainsi que leur mention selon la note : 'éliminé', 'admis', ou 'redoublant'

TP1-Tsql : Eléments de base

- 1- Créer une base de données avec la table client (idc, nom,prenom,ville) , l'id du client est auto incrémenté.
- 2- Insérer des lignes à la table client tant que le nombre de lignes déjà insérée est inférieure à 6.
- 3- Afficher les clients avec leurs villes en majuscules
- 4- -Afficher le nom et le prenom des clients de safi s'il existent et 'aucun client de safi' sinon

II- Procédures Stockées

Une procédure stockée (stored procedure) est une suite d'instructions SQL stockées dans la base de données et pouvant être exécutée par appel de son nom avec ou sans paramètres.

Syntaxe :

```
CREATE PROC[EDURE] procedure_name [ parametres]
As
Begin
Instruction ou Bloc d'instructions
end
```

Avec :

- Pour chaque paramètre, on écrit : @nompara type [=valeur par défaut] [output]
- Output indique que le paramètre est un paramètre de retour renvoyé par la procédure

Exemple : écrire une procédure qui permet de calculer la factorielle d'un nombre .Exécuter la procédure pour trouver la factorielle de 5

Remarque : pour exécuter une procédure : **exec nomproc parametres**

Solution :

```
/*Procédure stockée*/
/* factorielle sans parametre de retour*/
create proc factorielle @n int
as
begin
declare @f int;
set @f=1;
while(@n>=1)
begin
set @f=@f*@n;
set @n-=1;
end
select @f as 'factorille'
end

exec factorielle 5;
go;
```

```

/* factorielle avec parametre de retour*/

create proc factorielle2 @n int,@f int output
as
begin
set @f=1;
while(@n>=1)
begin
set @f=@f*@n;
set @n=@n-1;
end
end
/*execution*/
declare @fact int;
exec factorielle2 5,@fact output
select @fact;

```

Remarques :

- Pour supprimer une PS : **Drop Proc procedure_name**
- Pour Modifier une PS : **Alter Proc procedure_name paramètre as sql_stat**

TP2-Tsql : Procédures stockées

Soit le schéma relationnel suivant :

Personne(nomp,nomp,prenomp,ville)

Voiture(matricule,model,annee,#nump)

Voyage(#nump,#matricule,ville,datev)

- 1- Créer la bd
- 2- Créer une procédure createtables qui permet de construire les trois tables en les supprimant s'ils existent avant leur création (utiliser sys.tables)
- 3- Créer une procédure ajoutvoyage qui permet d'insérer un voyage en vérifiant l'intégrité référentielle
- 4- Créer une procédure nbrevoyage qui permet de calculer le nombre de voitures d'une personne sachant son numéro.

TP3-Tsql : Procédures stockées

Soit la base de données :

Client(idc,nom, prenom, ville)

Commande(idcmd,#idc,#idp,datecmd,qtitecmd)

Produit(idp,libelle,prixunitaire) ;

- 1- Créer la base avec des jeux de test
- 2- Créer les procédures AjoutClient , AjoutPProduit et Commander qui permettent d'ajouter une ligne dans les tables client, produit et commande respectivement.

- 3- Utiliser la procedure AjoutClient pour inserer un nouveau client . Peut –on changer l’ordre des paramètres de la procédure lors de son exécution ? Comment ?
- 4- Créer une procédure qui permet de lister des clients dont le nom commence par une lettre spécifiée par l'utilisateur.
- 5- Créer une procédure qui affiche la liste des commandes effectuées entre deux dates passées en paramètres. En plus si le nombre de commandes est supérieur à 100, afficher 'Période rouge'. Si le nombre de commande est entre 50 et 100 afficher 'Période jaune' sinon afficher 'Période blanche'
- 6- SP qui stocke dans une nouvelle table temporaire les 5 meilleures commandes
- 7- Créer une procédure stockée qui permet d’afficher pour chaque client , une« observation » avec valeur= ‘client classe A’, ou ‘client classe B’, ‘Classe C’ ou ‘nbre commande <2’ en fonction du nombre de commande qu’il a effectué cette année (classe A >10, classe B entre 10 et 5, classe C entre 5 et 2)

Résumé : les procédures stockées ne sont que des segments de code qui portent un nom, et qui sont précompilés, ce qui améliore et leur fiabilité, et leur performance.

III- Les fonctions utilisateur : (UDF- User Define Function ou Function Définie par l'Utilisateur)

Une fonction que le concepteur de la base écrit pour des besoins de traitement au sein des requêtes et du code des procédures stockées ou des triggers. Elle fait donc partie intégrante de la base où elle est considérée comme un objet au même titre qu'une table, une vue, un utilisateur ou une procédure stockée. Il existe deux grands types de fonctions : celles renvoyant une valeur et celles renvoyant un jeu de données (table).

1. Fonctions scalaire : retourne une valeur

Syntaxe :

```
CREATE FUNCTION function_name ( @nomparametre type [=default] ,... )
RETURNS type_valeur_retour
[ AS ]
BEGIN
function_body
RETURN valeur
END [ ; ]
```

Remarque:

Tous les types de données peuvent être retournés par une fonction scalaire hors mis timestamp, table, cursor, text, ntext et image

Exemple 1 :

```
CREATE FUNCTION f1 (@f int)
RETURNS int
AS
BEGIN
return (select count(*) from stagiaire where n_fil=@f)
END
```

Exemple 2 : fonction qui retourne la factorielle d'un nombre passé en paramètre

```
CREATE FUNCTION Factoriel (@n int) RETURNS bigint
AS
BEGIN
declare @f bigint, @i int
select @f=1, @i=1
while (@i<=@n)
begin
select @f=@f*@i, @i=@i+1
end
RETURN @f
END
Select dbo.Factoriel(6) as Factoriel;
```

```

ou
Declare @r bigint
Set @r=dbo.Factoriel(15)
Print cast(@r as varchar)

```

Remarque 2 : Le dbo est un utilisateur qui a les autorisations pour exercer toutes les activités dans la base de données. Tout objet créé par un membre du rôle serveur fixe sysadmin appartient à dbo automatiquement.

2- Fonction qui retourne une table (Inline Table-valued Functions) :

Les fonctions table retourne comme résultat une table qui est le résultat d'une instruction SELECT.

Syntaxe :

```

CREATE FUNCTION function_name ( @nomparametre type [=default] ,... )
RETURNS TABLE
[ AS ]
RETURN [ ( ) select_stmt [ ] ] [ ; ]

```

Exemple : CREATE FUNCTION recommander_stock (@Id int, @seuil int)
 RETURNS TABLE
 AS
 RETURN (SELECT * FROM Stock WHERE Id_Stock = @Id AND Quantite <
 @Seuil)

3- Fonction table multi-instruction : Multistatement Table-valued Functions

Ce type de fonction retourne une table on multi-instruction

Syntaxe :

```

CREATE FUNCTION function_name ( @nomparametre type [=default] ,... )
RETURNS @return_variable TABLE < table_type_definition >
[ AS ]
BEGIN
function_body
RETURN
END [ ; ]

```

Exemple 1 :

```

CREATE FUNCTION table_multi (@Id int)
RETURNS @variable TABLE (Id_Stock int, Quantite int, Nom_Entrepos varchar(25))
AS
BEGIN
SELECT @variable = (SELECT Id_Stock, Quantite, Nom_Entrepos
FROM Entrepos E INNER JOIN Stock S
ON E.Id_Entrepos = S.Id_Entrepos

```

```
WHERE S.Id_Stock = @Id)
RETURN
END
```

Exemple 2 : une fonction table à instructions multiples

```
Create FUNCTION NoteStg2 (@idstg int)
RETURNS @Resultat Table (Matiere varchar(50), Note decimal(4,2))
AS
begin
insert into @Resultat Select 'Nom : ' + nom, Null from stagiaire where idstg=@idstg
insert into @Resultat Select libelle, avg(note)
From Matiere inner join note on Matiere.idmat=note.idmat
Where idstg=@idstg Group by Matiere.idmat, libelle
insert into @Resultat select 'Moyenne : ', Moyenne from stagiaire where idstg=@idstg
RETURN
end
Go
Select * from dbo.NoteStg2(1);
Go
```

Remarque : Les fonctions table et table multi-instructions sont différentes dans le sens où le format de retour est différent. En effet, la fonction table retourne la solution d'une requête SELECT, alors que la fonction table multi-instruction, retournera une variable de type table, contenant l'instruction SELECT opérée par la fonction.

Remarque :

- Modifier une fonction : Il suffit de remplacer le mot clé **CREATE FUNCTION** par le mot clé **ALTER FUNCTION**.
- Suppression d'une fonction : On utilise la commande drop comme suit : **Drop Function nom_fonction**

TP4-Tsql : Fonctions Utilisateur

On suppose qu'on a les tables suivantes :

Stagiaire (ids,nom,prenom,daten)

Matiere(idm,libelle,mh)

Notation(idn,#ids,#idm,note)

1. Créer une fonction qui permet de retourner les notes des matières pour un stagiaire donné.
2. Écrire une fonction qui retourne pour un stagiaire donnée, la liste des modules avec la moyenne des notes pour chaque module.
3. Ecrire une fonction qui retourne la moyenne pour un stagiaire dont id donnée en paramètre

IV- Triggers (Déclencheurs)

Un triggers est une Forme évoluée de règles utilisées pour renforcer l'intégrité de la base de données. Autrement dit, les triggers sont des contraintes d'intégrité personnalisée. Les triggers sont un type particulier de procédure mémorisée. En effet, les triggers sont attachés à des tables, Ils ne peuvent pas être appelés explicitement dans les applications. Les triggers sont déclenchés automatiquement par le noyau SQL à chaque événement (insert, update ou delete) sur la table qui les supportent.

1- Les déclencheurs for ou after :

Syntaxe :

```
CREATE TRIGGER nom_trigger ON nom_table FOR INSERT[,update][,delete]
AS
bloc d'instruction SQL
```

Principe de fonctionnement.

Deux tables virtuelles (variable local temporaire dans le programme) sont créées au moment de la MAJ des données sur une table (**INSERTED**, **DELETED**) en lecture seul, Elles sont destinées à contenir les lignes de la table sur lesquelles ont été effectuées des opérations.

Les tables INSERTED et DELETED peuvent être utilisées par le trigger pour déterminer comment le traitement doit se dérouler. Ce traitement est à écrire par le développeur.

- **Cas de suppression d'une ligne de table (Delete) :**
La/les lignes supprimées sont placées dans la table temporaire DELETED et supprimées de la table réelle;
- **Cas de création d'une ligne de table (Insert) :** La/les lignes nouvelles sont placées dans la table temporaire INSERTED et dans la table réelle;
- **Cas de modification d'une ligne de table (Update) :** La/les lignes avant modification sont placées dans la table temporaire DELETED et la/les lignes après modification sont placées dans la table temporaire INSERTED et dans la table réelle.

Exemple: on suppose qu'on a dans notre base de données, la table notation (idn,#idModule,#idStagiaire, note). On veut limiter le nombre de note par stagiaire par Module à 3 .créer un trigger qui permet de gérer cette contrainte:

Solution :

```
alter TRIGGER LimitNote ON notation
FOR INSERT
AS
begin
if (select count(*)
from notation n,inserted i
where n.idstagiaire=i.idstagiaire and n.idmodule=i.idmodule)>3
begin
RAISERROR('nombre limite 3 notes par matière insertion annuler',15,1)
rollback /*sinon le message s'affiche et l'ajout s'effectue*/
```

```

end
end
insert into notation values (1,1,12);
insert into notation values (1,1,2);
insert into notation values (1,1,10);
insert into notation values (1,1,15) ;
select * from notation

```

Exercice :

Créer deux tables de meme schéma stagiaire et archive .Créer un trigger qui permet de sauvegarder, dans la table archive , chaque stagiaire supprimé

2. Déclencheurs LMD de type INSTEAD OF (au lieu de):

Les déclencheurs INSTEAD OF sont exécutés à la place de l'action de déclenchement (UPDATE, DELETE et INSERT). Ils sont exécutés avant toute contrainte, ce qui leur permet d'effectuer un prétraitement qui complète les actions des contraintes.

Remarque : Chaque table ou vue peut disposer d'un seul déclencheur INSTEAD OF par action de déclenchement.

Exemple : écrire un trigger qui permet d'afficher 'non' après demande de suppression d'une ligne de la table notation. La suppression ne doit pas s'effectuée.

Exemple (Instead of): supposons qu'on a deux tables module et notation. Si on veut supprimer des modules, on aura un problème lié à la contrainte d'intégrité référentiel.

Créer un trigger qui permet de supprimer les modules en supprimant les notes liés à ces modules en premier.

```

CREATE TRIGGER Supp
ON Matière
INSTEAD OF DELETE
AS
begin
Delete Form Notation Where Idm in (select idm from deleted)
Delete From module Where Idm in (select idm from deleted)
end
Go

```

Remarque 1:

- Suppression d'un triggers: **DROP TRIGGER nom_trigger**
- Modifier un triggers : la même syntaxe de **Create**, en remplaçant **Create** par **Alter**
- Désactiver un trigger : **alter table nomtable DISABLE TRIGGER nomTrigger**

Remarque 2: La suppression d'une table entraîne la destruction de ses triggers

Remarque 3: Avec SQL 2005 et plus, on peut avoir des déclencheurs sur LDD (Langage de Définition de Donnée) comme (Create, Alter, Drop) et à certaines procédures stockées système qui effectuent des opérations de type LDD

Résumé :

Fonction	Déclencheur AFTER	Déclencheur INSTEAD OF
Applicabilité	Tables	Tables et vues
Quantité par table ou vue	Plusieurs par action de déclenchement (UPDATE, DELETE et INSERT)	Un par action de déclenchement (UPDATE, DELETE et INSERT)
Références en cascade	Aucune restriction	Non autorisé sur des tables cible de contraintes d'intégrité référentielle en cascade
Exécution	Après : Traitement des contraintes Création de tables inserted et deleted	Avant : Traitement des contraintes Après : création de tables inserted et deleted

TP5-Tsql : les Triggers

soit le schéma relationnel suivant

Employe (Matricule, nom, prenom, telephone, adresse, salaire)

Division (idDiv, libelle)

travailler (idT, #Matricule, #iddiv)

1. Créer la base de données avec les champs et les contraintes pk et fk
2. Créer un trigger qui affiche les lignes insérées pour insert sur la table Employé
3. Créer un trigger qui affiche le nouveau numéro de téléphone ainsi que l'ancien lorsqu'une modification de la colonne téléphone est effectuée sur un tuple de la table Employe
4. Créer un trigger qui empêche la modification du libelle d'une division
5. Refaire l'implémentation des contraintes d'intégrité suivantes par des triggers:
 - i. Référentiel (clé étranger)
 - ii. De domaine ($2500 \leq \text{salaire} \leq 20000$)
6. Implémenter la suppression en cascade

V- Utiliser les curseurs :

Définition :

Un curseur est un groupe d'enregistrements résultat de l'interrogation d'une base de données. L'intérêt d'utiliser des curseurs est de pouvoir faire des traitements ligne par ligne chose qui n'est pas possible avec une requête SQL simple

Syntaxe :

1. déclarer un curseur :

**Declare nom_curseur [scroll] Cursor
For Requête**

2. ouvrir un curseur : **Open nom_curseur**

3. lire un enregistrement à partir d'un curseur :

Fetch First|Next|Prior|Last from nom_curseur into variable1, variable2,..

- Atteindre le premier enregistrement du curseur : first
- Atteindre l'enregistrement du curseur suivant celui en cours:next
- Atteindre l'enregistrement du curseur précédent celui en cours : Prior
- Atteindre le dernier enregistrement du curseur : Last

Remarque 1 :

- Atteindre l'enregistrement se trouvant à la position n dans le curseur : **Fetch absolute n from nom_curseur into variable1, variable2,...**
- Atteindre l'enregistrement se trouvant après n positions de la ligne en cours : **Fetch Relative Num_Ligne from nom_curseur into variable1, variable2,...**

Remarque 2 : La variable système **@@fetch_status** est utilisée pour détecter la fin du curseur. Tant que cette variable a la valeur 0, on a pas encore atteint la fin du curseur.

4. Fermer un curseur : **Close nom_curseur**

5. Libérer les ressources utilisées par un curseur : **Deallocate Nom_Curseur**

Exemple : soit la table article(numA,libelle,PU) . Afficher la liste des articles sous la forme : L'article Numéro ... portant la désignationcoûte

Solution :

```
Declare @a int, @b Varchar(10), @c float
Declare Cur_ListeArticle Cursor for Select NumA, libelle,PU from article
Open Cur_ListeArticle
Fetch Next from Cur_ListeArt into @a,@b,@c
While @@fetch_status=0
Begin
Print 'L'article numéro ' + convert(varchar,@a) + ' portant la
désignation ' + @b+ ' coûte ' + convert(varchar,@c)
Fetch Next from Cur_ListeArt into @a,@b,@c
```

End
 Close Cur_ListeArticle
 Deallocate Cur_ListeArticle

REMARQUE : Types de curseurs :

- CURSEURS AVANT UNIQUEMENT (sans scroll) : permettent de lire les lignes dans l'ordre (de la première jusqu'à la dernière) .Utilise Next seulement

-CURSEURS DE DEFILEMENT : (scroll)
 Toutes les options de FETCH sont autorisés

TP6-Tsql : Les Curseurs

TP : soit la le schéma relationnel suivant :

Article(numA, libelle, PU,QtiteStock)

Commande(numCmd,DateCmd)

LigneCommande(#numCmd,#numA,QtiteCmd)

1. Ecrire un programme qui, pour chaque commande : Affiche le numéro et la date sous la forme :
 Commande Effectuée le

Il permet aussi d'afficher la liste des articles associés et le montant de cette commande

2. Ecrire un programme qui pour chaque commande vérifie si cette commande a au moins un article. Si c'est le cas affiche son numéro et la liste de ses articles sinon affiche un message d'erreur
 'Aucun article pour la commande , Elle sera supprimée' et supprime cette commande

VI- Sauvegarde et restauration des bases de données

Pour réduire le risque de perte catastrophique de données, vous devez sauvegarder vos bases de données pour conserver les modifications apportées à vos données de façon régulière. Une stratégie de sauvegarde et de restauration correctement planifiée permet de protéger les bases de données contre la perte de données provoquée par différentes défaillances.

1- sauvegarde complète

Sauvegarde de données qui contient toutes les données d'une base de données ainsi qu'une partie suffisante du journal

Syntaxe : BACKUP DATABASE NomBD TO DISK = 'c:\...\NomBD.bak'

2- sauvegarde différentielle

Sauvegarde de données basée sur la dernière sauvegarde complète d'une base de données complète et qui contient uniquement les données ayant changé depuis cette base.

Syntaxe : BACKUP DATABASE NomBD TO DISK = 'c:\...\NomBD.bak' with differential

3- sauvegarde de fichier journal

Sauvegarde des journaux des transactions qui inclut tous les enregistrements des journaux qui n'ont pas été sauvegardés lors d'une sauvegarde de fichier journal précédente. (mode de récupération complète)

Syntaxe : BACKUP log NomBD TO DISK = 'c:\...\NomBD.trn'

Remarque : mode de récupération

Propriété de base de données qui contrôle la maintenance du journal des transactions sur une base de données. Il existe trois modes de récupération : **simple**, **complète** et **utilisant les journaux de transactions**. Le mode de récupération de base de données détermine les spécifications de sauvegarde et de restauration.

Restauration des données :

On restaure la dernière sauvegarde complète, la dernière sauvegarde différentielle et le dernier fichier journal.

Remarque : On peut utiliser le mode assistant pour sauvegarder ou restaurer une base de données. On peut même l'utiliser pour récupérer le script en cas d'oubli.

TP :

Créer une table dans votre base de données.

- 1- Remplir la table avec 10 lignes puis effectuer une sauvegarde complète.
- 2- Remplir la table avec 10 nouvelles lignes. Effectuer une sauvegarde différentielle.
- 3- Ajouter 10 lignes encore à votre table puis effectuer une sauvegarde des fichiers journaux et transaction.
- 4- On simule une panne : on suppose qu'on a exécuté par erreur le code qui supprime la table de la base de données. Donner le résultat si :
 - 4.1- on exécute directement : `select * from table ;` //nom de votre table
 - 4.2- on effectue une restauration de la sauvegarde complète. Puis, on exécute : `select * from table ;`
 - 4.3- on effectue une restauration de la sauvegarde différentielle. Puis, on exécute : `select * from table ;`. Est-ce qu'on peut effectuer la restauration de la sauvegarde différentielle seule ?!
 - 4.4- on effectue une restauration de la sauvegarde de fichier journal. Puis, on exécute : `select * from table`. Pour faire cette restauration, qu'est-ce qu'on doit changer?

Révision :

Examen fin formation 2010 th :

Une compagnie aérienne désire gérer l'ensemble de ses vols qu'elle met en service ainsi que les pilotes et les avions qui effectuent ces vols.

Le schéma de la base de données est le suivant :
 constructeur(idconstructeur , nomconstructeur);

pilote(idpilote , nompilote , prenompilote)

Aeroport(idaeroport , nomaeroport , nomvilleDesservie)

typee(typeavion , capacite , idconstructeur)

avion(numAvion , typeAvion ,baseaeroport)

vol(numv ,aeroportDepart, aeroportArrivee , heureDepart ,heureArrivee);

affectation(numvol, datevol, numAvion , idpilote);

0-Ajouter # à côté des clés étrangères, et souligner les clés primaires

1- Ajouter le champ ville à la table pilote

2- donner le nom des pilotes planifiés pour des vols sur A320

3- donner le nom des pilotes planifiés pour des vols sur A320 et qui habitent dans la ville de localisation d'un A320

4-pour chaque ville desservie, donner la moyenne,le minimum et le maximum des capacités des avions qui sont localisés dans un aéroport desservant cette ville.

5- dans la table vol, créer un trigger en insertion qui vérifie que la ville d'arrivée est différente de la ville de départ et que l'heure d'arrivée est postérieure à l'heure de départ. Dans le cas contraire, on annule l'insertion

6- créer une procédure stockée 'Planning qui affiche pour un nom et un prénom de pilote passés en paramètres, les numéros de vol,avec les aéroports de départ et d'arrivée, classés par date et heure de départ

Révision : Examen th 2007

Vous participez à la réalisation d'un logiciel permettant de gérer les comptes bancaires d'un particulier

Dans un premier temps, l'utilisateur définit les différents comptes à gérer en déclarant pour chacun le numéro de compte, le nom de la banque ,...

L'utilisateur saisit ensuite chacune des opérations réalisées sur l'un de ces comptes. Une opération correspond à un débit (paiement par chèque de la facture de téléphone , retrait d'espèce à l'aide d'une carte bancaire,...) ou à un crédit (dépôt à la banque de chèque reçu par exemple).

Chaque fin de mois, l'utilisateur reçoit un relevé de compte (un par compte). Il effectue alors le rapprochement entre les opérations qu'il saisies et celles constatées par la banque :

- Pointage des opérations saisies apparaissant sur le relevé. Ce pointage consiste à mémoriser le fait qu'une opération a été constatée par la banque. L'opération est dite 'pointée' et le numéro du relevé est mémorisé
- Ajout des opérations non saisies apparaissent sur le relevé (réception d'un virement correspondant au salaire mensuel par exemple). Ces opérations sont automatiquement pointées puisqu'elles ont été constatées par la banque.

Les opérations permettent de calculer le solde d'un compte, différence entre la somme des opérations de crédit et la somme des opérations de débit. En fait, chaque compte possède deux soldes :

- Le solde réel qui prend en considération toutes les opérations saisies par l'utilisateur , qu'elles aient été constatées ou non par la banque
- Le solde en banque qui prend en considération que les opérations constatées par la banque c'est-à-dire ayant fait l'objet d'un pointage à partir d'un relevé

Le calcul des soldes des comptes conduit à additionner un grand nombre d'opérations, et ce depuis la mise en service de l'application. Pour éviter ces calculs, il a été décidé de mémoriser les soldes réels et 'en banque' dans la table compte.

Dés lors se pose le problème de la cohérence entre les opérations et le solde des comptes.

Solution adoptée est le recours à un ensemble de procédures stockées et de déclencheurs destinés à maintenir à jour les soldes dans la table compte

Schéma de la base de données :

Banque(Bid,Bnom)

Compte(Cid,Cnuméro,Clibelle,Csoldereel,csoldeBanque,#Cbanque)

Opération(Oid,Odate,Omontant,Osens,#Oreleve,#Ocompte)

Releve(Rid,Rdate,#compte)

Remarque :

- Csoldereel et Csoldebanque représentent respectivement le solde réel et le solde en banque du compte
- Osens est un caractère mémorisant la nature de l'opération(D : debit, C : credit)
- Oreleve contient le numéro de relevé Rid ayant permis de pointer l'opération. Cet attribut contient la valeur NULL si l'opération n'a pas encore été pointée

Rédiger l'ensemble des procédures stockées et des triggers nécessaires à la gestion des informations calculées Csoldereel et Csoldebanque :

- Procédure MAJSOLDEBANQUE
- Procédure MAJSOLDEREEL
- Trigger insertion operation
- Trigger mise à jour opération
- Trigger suppression operation